

Contents

RGEE Security Assessment Report	1
Table of contents	1
Executive summary	2
Scope and methodology	2
Application architecture and trust boundaries	3
Security concepts used in this assessment	4
Threat model	5
Findings mapped to OWASP Top 10:2025	6
Controls already in place (positive findings)	9
Residual risks and operational recommendations	9
Part 2 — Detailed remediation and implementation guide	10
1. Exam session access tokens (A01-1) — detailed	10
2. CSRF protection (A06-3) — detailed	11
3. Security headers middleware (A02-1) — detailed	12
4. Production security gate (A02-3, A07-1) — detailed	13
5. Input length limits (A05-4) — detailed	13
6. Session fixation fix (A07-2) — detailed	14
7. Broken HTML hint endpoint (A10-1) — detailed	14
8. Analysis agent hardening (A01-3, A02-4) — detailed	14
9. HTTPS-only session cookies (A02-2) — detailed	15
10. Automated security tests — detailed	15
File change index	15
Deployment checklist (production)	15
Verification and test evidence	16
References	16

RGEE Security Assessment Report

Application: Rubric-Guided Essay Exam (RGEE)

Assessment date: 22 May 2026

Version assessed: post-remediation codebase

Primary framework: OWASP Top 10:2025

Supplementary standards: OWASP ASVS 4.0, OWASP Cheat Sheet Series, Mozilla Web Security Guidelines, CWE Top 25, NIST SP 800-63B (authentication guidance)

Table of contents

1. Executive summary
 2. Scope and methodology
 3. Application architecture and trust boundaries
 4. Security concepts used in this assessment
 5. Threat model
 6. Findings mapped to OWASP Top 10:2025
 7. Controls already in place (positive findings)
 8. Residual risks and operational recommendations
 9. Part 2 — Detailed remediation and implementation guide
 10. Verification and test evidence
 11. References
-

Executive summary

RGEE is a server-rendered web application that lets students take rubric-guided essay exams while instructors review sessions, manage Together.ai API credentials, and run embedding-based question analysis. The application integrates with external LLM APIs (Together.ai) and optionally a separate analysis microservice (RGEE_Analysis_Agent).

This assessment reviewed the codebase against **OWASP Top 10:2025**, documented vulnerabilities and design weaknesses, and implemented remediations in application code, templates, Docker configuration, and automated tests.

Metric	Before remediation	After remediation
Overall risk (typical classroom deployment)	High	Medium–Low
Overall risk (internet-facing + production LLM)	Critical	Medium
Automated tests passing	66	66
Critical findings open	3	0 (mitigated or fixed)
High findings open	4	0 (fixed)

The three most significant issues were:

1. **Broken access control (IDOR)** on student exam URLs — knowing an integer session ID was sufficient to read or modify another student’s exam.
2. **Missing CSRF protection** on all HTML forms — a malicious site could trick a logged-in instructor or an open student tab into submitting unwanted actions.
3. **Weak production defaults** — predictable session signing keys and auto-created instructor credentials suitable only for local development.

All implemented fixes are backward-compatible for development (with explicit environment flags) and enforce stricter behavior when `RGEE_PRODUCTION=1`.

Scope and methodology

In scope

Component	Path / description
Main web application	<code>app/main.py</code> , supporting modules
Security module	<code>app/security.py</code> (new)
Authentication	<code>app/instructor_auth.py</code>
Configuration	<code>app/config.py</code> , <code>.env.example</code>
Data layer	<code>app/database.py</code> (ORM usage review)
LLM integration	<code>app/llm_service.py</code> , <code>app/prompts.py</code>
API key storage	<code>app/together_credentials.py</code>
Templates / XSS surface	<code>templates/**</code> , <code>static/**</code>
Analysis sidecar	RGEE_Analysis_Agent/
Container layout	<code>docker-compose.yml</code>
Automated tests	<code>tests/security/</code> , <code>tests/general/</code>

Out of scope

- Penetration testing of production hosting (AWS, nginx, TLS certificate configuration)
- Together.ai platform security, Hugging Face model hosting
- Physical security, endpoint malware on student devices
- Formal compliance certification (FERPA, GDPR) — noted where relevant

Methodology

1. **Architecture review** — mapped routes, authentication boundaries, and data flows.
 2. **Static code analysis** — searched for injection sinks, missing auth checks, secret handling, error disclosure.
 3. **Threat modeling (STRIDE-lite)** — spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege per major feature.
 4. **OWASP Top 10:2025 mapping** — each finding tagged to the 2025 category (source).
 5. **Remediation implementation** — code changes with regression testing.
 6. **Verification** — `python -m pytest tests/ -q` (66 tests).
-

Application architecture and trust boundaries

High-level components

```
[Student browser]
| HTML forms, JavaScript, session cookie
v (HTTPS recommended)
[RGEE Main App - FastAPI / Uvicorn]
| Jinja2, SessionMiddleware, security middleware
| SQLAlchemy ORM -> SQLite (default)
+---> [Together.ai API] (LLM, Bearer token)
+---> [RGEE_Analysis_Agent :8010] (HTTP + shared secret header)
```

Trust boundaries

Boundary	What crosses it	Trust assumption
Browser <-> Main app	HTTP(S), cookies, form POST	Browser is user's; cookie must not be forgeable
Main app <-> Database	SQL via ORM	DB is trusted; app must not construct unsafe SQL
Main app <-> Together.ai	API key in Authorization header	Key is secret; only instructors configure it
Main app <-> Analysis agent	JSON over HTTP, optional X-RGEE-Analysis-Secret	Agent network should not be public
Instructor <-> Student data	Instructor login session	Only authenticated instructors see all sessions

Session model (important for understanding fixes)

RGEE uses **one signed session cookie** (`rgee_instructor`) for both:

- **Instructor authentication** (`instructor_ok` flag in session)
- **Student exam access tokens** (`exam_access_tokens` map in session)
- **CSRF tokens** (`csrf_token` in session)

The cookie is **signed** with `INSTRUCTOR_SESSION_SECRET` using Starlette's `SessionMiddleware`. Tampering with cookie bytes invalidates the signature. This is **not encryption** — do not store plaintext secrets in the session; only opaque tokens and flags are stored.

Security concepts used in this assessment

This section explains the security primitives referenced throughout the report so the team can reason about future changes consistently.

Access control and IDOR

Access control ensures users can only perform actions and read data they are authorized for. **Insecure Direct Object Reference (IDOR)** (CWE-639) occurs when the application exposes an internal identifier (e.g. `/exam/42/question`) and relies on the client to stay honest, without verifying the requester owns object 42.

Before fix: Session ID was a sequential integer in the URL. Any client could request `/exam/5/results` without proving they started or resumed exam 5.

Concept applied: Capability-based access via server-side session binding. After legitimate start/resume, the server stores a random token keyed by session ID inside the signed session cookie. Subsequent exam routes require that binding to exist.

Cross-Site Request Forgery (CSRF)

CSRF (CWE-352) tricks a victim's browser into sending an authenticated request to a site where they already have a session. Because browsers automatically attach cookies, the server may treat the request as legitimate.

Example attack (before fix): An instructor visits `evil.example` while logged into RGEE. A hidden form on the evil page POSTs to `/professor/together-credentials/save` with an attacker-controlled API key.

Concept applied: Synchronizer token pattern. Each session holds a random CSRF token. State-changing POSTs must include the token; the server compares it with `secrets.compare_digest()` (constant-time comparison prevents timing side channels).

Reference: OWASP CSRF Prevention Cheat Sheet.

Security headers and defense in depth

HTTP response headers instruct browsers to enable platform-level protections:

Header	Purpose
Content-Security-Policy (CSP)	Restricts which scripts, styles, fonts, and connections the page may load — mitigates XSS impact
X-Frame-Options: DENY	Prevents clickjacking by forbidding iframe embedding
X-Content-Type-Options: nosniff	Stops MIME-type confusion attacks
Referrer-Policy	Limits leakage of URL paths to third parties
Strict-Transport-Security (HSTS)	Forces HTTPS for future visits after first secure connection
Permissions-Policy	Disables unused browser features (camera, geolocation, etc.)

Reference: Mozilla Web Security Guidelines.

Session management and fixation

Session fixation (CWE-384) occurs when an attacker establishes a session ID before the victim logs in, then reuses that ID after the victim authenticates.

Concept applied: On successful instructor login, `request.session.clear()` destroys prior session data before setting `instructor_ok`, so the post-login session ID is fresh.

Reference: OWASP Session Management Cheat Sheet.

Password storage (instructor)

RGEE never stores plaintext instructor passwords. It stores:

- **Username:** SHA-256 hash of UTF-8 username (one-way)
- **Password:** PBKDF2-HMAC-SHA256 with 210,000 iterations and application salt

Verification uses `secrets.compare_digest()` to avoid timing leaks. This aligns with NIST SP 800-63B password verifier guidance.

Fail-closed production configuration

Fail-closed production gate: When `RGEE_PRODUCTION=1`, misconfiguration prevents startup rather than running with known-weak defaults — a **secure-by-default** deployment pattern.

Threat model

Assets

Asset	Sensitivity	Impact if compromised
Student essay answers	High	Academic integrity, privacy
Grades and rubric feedback	High	Fairness, privacy
Together API key	Critical	Financial abuse, data exfil via LLM
Instructor credentials	Critical	Full dashboard access
Session signing secret	Critical	Session forgery, account takeover
Performance logs	Medium	Student ID exposure to instructors (intended)

Threat actors

Actor	Capability	Primary goals
Anonymous internet user	HTTP client, can guess IDs	Access exams, burn LLM credits
Student (malicious)	Own browser, may know peer IDs	View/alter others' exams (before fix)
External attacker	Phishing, CSRF, XSS	Steal instructor session, swap API keys
Insider instructor	Legitimate access	View all student data (by design)

Attack scenarios considered

1. **Sequential ID enumeration** — iterate `/exam/1`, `/exam/2`, ... to harvest responses. **Mitigated** by exam access tokens.
2. **CSRF on instructor credential save** — swap Together API key. **Mitigated** by CSRF tokens.
3. **Default credential login** — `elliott/12345` on first boot. **Mitigated** in production; dev warning in logs.
4. **Forged session cookie** — without valid `INSTRUCTOR_SESSION_SECRET`. **Mitigated** by signed cookies; production requires unique secret.
5. **Direct analysis agent access** — POST to port 8010 without secret. **Mitigated** by `RGEE_ANALYSIS_AGENT_REQUIRE_SECRET` and localhost bind in Docker.

6. **LLM prompt injection via answers** — manipulate grading prompts. **Partially mitigated** (length limits); full mitigation is ongoing design work.
7. **Resume code brute force** — 5-char alphanumeric + known student ID. **Accepted risk**; recommend rate limiting at proxy.

Findings mapped to OWASP Top 10:2025

The 2025 OWASP Top 10 reorganizes categories to emphasize access control, misconfiguration, supply chain, and exceptional-condition handling. Below, each finding includes **severity**, **CWE** where applicable, **attack scenario**, and **status**.

A01:2025 — Broken Access Control

Access control enforces that users cannot act outside their permissions. Failures allow unauthorized disclosure, modification, or destruction of data.

A01-1 — Insecure Direct Object Reference on exam routes (Critical) — FIXED

Field	Detail
CWE	CWE-639 (Authorization Bypass Through User-Controlled Key)
Affected routes	GET/POST /exam/{session_id}/* (question, answer, hint, hint-json, results, client-timing)
Root cause	Authorization decision relied solely on knowledge of <code>session_id</code> ; no proof the browser had legitimately started or resumed that exam

Attack scenario: Student A completes exam and mentions “I’m on question 3.” Student B navigates to /exam/{A's_id}/question, reads prompts, submits answers, or triggers paid LLM grading calls.

Remediation concept: Object-level authorization via session-stored capability. See Part 2 §1 for implementation detail.

Verification: tests/security/test_controls.py::test_exam_routes_require_session_access_token

A01-2 — Instructor route protection (Low) — OK Instructor pages use `_instructor_login_redirect()` checking `instructor_ok` in session. Open redirect via `next` parameter is blocked by `_safe_instructor_next_url()` (must start with /, not //).

A01-3 — Analysis agent missing authentication (High) — FIXED

Field	Detail
CWE	CWE-306 (Missing Authentication for Critical Function)
Affected	POST /internal/v1/analyze on <code>RGEE_Analysis_Agent</code>
Root cause	When <code>RGEE_ANALYSIS_AGENT_SECRET</code> was empty, <code>_check_secret()</code> returned without error

Attack scenario: Attacker on the same network as Docker host POSTs analysis jobs, causing CPU load and arbitrary persistence to agent DB.

Remediation: Optional strict mode `RGEE_ANALYSIS_AGENT_REQUIRE_SECRET=1`; Docker compose sets shared secret and binds `127.0.0.1:8010`.

A02:2025 — Security Misconfiguration

A02-1 — Missing HTTP security headers (Medium) — FIXED Browsers lacked CSP, clickjacking protection, and MIME sniffing guards. See Part 2 §3.

A02-2 — Session cookie not HTTPS-only (Medium) — FIXED `SessionMiddleware(https_only=False)` allowed cookie transmission over cleartext HTTP. Fixed via `INSTRUCTOR_SESSION_HTTPS_ONLY` environment variable.

A02-3 — Default session signing secret (High) — MITIGATED Predictable default in `app/config.py` enables cookie forgery if deployed unchanged. Production startup validation rejects the default when `RGEE_PRODUCTION=1`.

A02-4 — Analysis agent exposed on all interfaces (Medium) — FIXED Docker published `8010:8010` on `0.0.0.0`. Changed to `127.0.0.1:8010:8010` so only the host (and containers on the bridge network) can reach it.

A02-5 — Default instructor credentials (Critical) — MITIGATED First-run creation of `instructor_credentials.json` with known password hashes for `elliott/12345`. Blocked when `RGEE_PRODUCTION=1`; development mode logs a explicit warning.

A03:2025 — Software Supply Chain Failures

A03-1 — No automated dependency vulnerability scanning (Low) — ACCEPTED Dependencies are listed in `requirements.txt` but CI does not run `pip-audit` or equivalent. **Recommendation:** add GitHub Dependabot or OWASP Dependency-Check to `.github/workflows/`.

A03-2 — Third-party CDN scripts (Low) — PARTIALLY MITIGATED KaTeX and html2pdf load from CDNs. CSP `script-src` allowlists those hosts. **Recommendation:** add Subresource Integrity (SRI) hashes when upgrading CDN versions.

A04:2025 — Cryptographic Failures

A04-1 — Static PBKDF2 salt (Low) — ACCEPTED Single application salt `rgee-instructor-v1` for all deployments. Per-user salts would require credential file schema migration. Risk is reduced because stored values are one-way hashes, not reversible passwords.

A04-2 — API key storage (Low) — OK Together keys prefer OS keyring, then Fernet-encrypted file; UI shows only masked suffix. Instructor-only routes gate write access (plus CSRF after fix).

A04-3 — SQLite not encrypted at rest (Low) — ACCEPTED Encrypt the host volume or use managed DB with encryption for production.

A05:2025 — Injection

A05-1 — SQL injection (—) — OK SQLAlchemy ORM with bound parameters throughout request handlers. Migrations use static SQL with named parameters.

A05-2 — Cross-site scripting / XSS (—) — OK Jinja2 auto-escaping enabled; user content rendered with `{{ var }}` not `|safe`. JSON embedded in scripts uses `|tojson`. Client-side hint rendering uses `textContent` in carousel code.

A05-3 — LLM prompt injection (Medium) — PARTIAL Student answers and professor domain text are passed to LLM prompts. Keyword heuristics exist for hint paths only. **Partial fix:** server-side length limits reduce abuse surface. **Future work:** structured prompt templates with strict role separation, output schema validation.

Reference: OWASP LLM Prompt Injection.

A05-4 — Unbounded input / resource exhaustion (Medium) — FIXED No server-side max length on `professor_domain` or `answer` allowed large payloads → DB bloat and LLM token cost. Limits: 8,000 / 50,000 / 256 characters.

A06:2025 — Insecure Design

A06-1 — Single shared instructor account (Medium) — ACCEPTED No role-based access control or per-instructor audit trail. Acceptable for current single-instructor deployment model.

A06-2 — Weak resume code entropy (Medium) — ACCEPTED 5 alphanumeric characters ($\sim 36^5$, about 60 million combinations). Combined with student ID, practical guessing requires many online attempts — mitigate with **rate limiting** at reverse proxy (not implemented in app).

A06-3 — Missing CSRF protection (High) — FIXED See Part 2 §2.

A07:2025 — Authentication Failures

A07-1 — Default / weak instructor credentials (Critical) — MITIGATED See A02-5.

A07-2 — Session fixation on instructor login (Medium) — FIXED Session cleared before setting authenticated flag.

A07-3 — Password verification implementation (—) — OK PBKDF2 with 210k iterations; constant-time compare.

A07-4 — No login rate limiting (Medium) — ACCEPTED Recommend `nginx limit_req` or Cloudflare rate rules on `/professor/login`.

A08:2025 — Software or Data Integrity Failures

A08-1 — No SRI on CDN assets (Low) — ACCEPTED

A08-2 — Nominated exam publishing (—) — OK Intended instructor feature; protected by instructor auth + CSRF.

A09:2025 — Security Logging and Alerting Failures

A09-1 — Error handling (—) — OK Global handlers return generic HTML; `logger.exception` records details server-side. Tests assert no tracebacks in responses (`tests/security/test_http.py`).

A09-2 — No security alerting (Low) — ACCEPTED No SIEM integration for 403 spikes or LLM cost anomalies.

A09-3 — PII in performance log (Low) — ACCEPTED By design for instructor/dev visibility.

A10:2025 — Mishandling of Exceptional Conditions

A10-1 — Broken fallback hint endpoint (Medium) — FIXED `POST /exam/{id}/hint` referenced undefined variables (`mode_key`, `selected_hint_text`, `query_text`), causing 500 errors for users without JavaScript. Simplified to call `generate_safe_hint()` only.

A10-2 — Unhandled exception disclosure (—) — OK Catch-all handler prevents stack trace HTML.

Controls already in place (positive findings)

Documenting existing good practices helps the team preserve them during future development.

Control	Location	Notes
ORM-only database access	<code>app/database.py</code> , route handlers	Reduces SQL injection risk
Jinja2 auto-escaping	All templates	Default XSS protection
Together API errors sanitized	<code>app/main.py</code> exception handler	No API key leakage in HTML
Open redirect prevention	<code>_safe_instructor_next_url()</code>	Instructor login <code>next</code> param
Signed session cookies	Starlette <code>SessionMiddleware</code>	Integrity of session data
<code>.gitignore</code> for secrets	<code>.env</code> , credential files	Prevents accidental commit
Security test CI job	<code>.github/workflows/python-tests.yml</code>	Runs <code>tests/security/</code>
Instructor password hashing	<code>app/instructor_auth.py</code>	PBKDF2, not plaintext
API key never shown in full	<code>together_credentials_snapshot()</code>	Masked suffix only

Residual risks and operational recommendations

Priority	Recommendation	Rationale
P0	Deploy with HTTPS + <code>RGEE_PRODUCTION=1</code> + unique secrets	Foundational transport and config security
P0	Rotate instructor password before any public URL	Default dev credentials must not reach production
P1	Rate-limit login, exam start, resume at reverse proxy	Brute force, LLM cost abuse
P1	Monitor Together.ai billing / usage alerts	Production LLM mode is a financial attack surface
P2	Add <code>pip-audit</code> to CI	Supply chain visibility (A03)
P2	Add SRI attributes to CDN script tags	Integrity verification (A08)
P3	Harden LLM prompts against injection	Partial coverage today

Priority	Recommendation	Rationale
P3	Consider per-deployment PBKDF2 salt migration	Minor cryptographic hardening

Part 2 — Detailed remediation and implementation guide

This section explains **what was changed**, **why**, **how it works**, and **which security concept** each change implements. Use it as onboarding material for developers maintaining RGEE.

1. Exam session access tokens (A01-1) — detailed

Problem (technical)

Exam routes used a **reference identifier** (`session_id` integer) as the sole authorization mechanism. In security terms, the URL parameter was both an identifier and an authorization credential — a classic **IDOR** anti-pattern (OWASP IDOR).

Security concept

Capability-based access control via server-side session state:

- A **capability** is an unguessable token that proves the holder previously completed a legitimate **start** or **resume** flow.
- The capability is stored **server-side** inside the signed session cookie, not in the URL.
- Knowing the integer `session_id` alone is insufficient.

This follows the principle of **defense in depth**: even if session IDs leak (browser history, referrer logs, shoulder surfing), an attacker still needs the victim's session cookie.

Implementation

New module: `app/security.py`

```
EXAM_ACCESS_SESSION_KEY = "exam_access_tokens"

def grant_exam_access(request, session_id):
    token = secrets.token_urlsafe(32) # 256 bits of entropy
    tokens = request.session.get(EXAM_ACCESS_SESSION_KEY) or {}
    tokens[str(session_id)] = token
    request.session[EXAM_ACCESS_SESSION_KEY] = tokens

def require_exam_access(request, session_id):
    # Raises HTTP 403 if no token stored for this session_id
```

Grant points (capability issued):

Route	When
POST /exam/start	After new session committed
POST /exam/start-nominated	After nominated session created
POST /resume	After student ID + exam code validated

Enforcement points (capability required):

Route	Method
/exam/{id}/question	GET
/exam/{id}/answer	POST
/exam/{id}/hint	POST
/exam/{id}/hint-json	POST
/exam/{id}/results	GET
/exam/{id}/client-timing	POST

Order of checks: Load session from DB → 404 if missing → `require_exam_access()` → 403 if cookie lacks token → proceed.

User-visible behavior

- Normal flow unchanged: student starts exam, browser receives cookie with token, all subsequent pages work.
- Attacker without the cookie: **403 Forbidden** with message to start/resume properly.
- Resume still requires **student ID + exam code** — the stronger gate before a capability is granted.

Limitations (team should know)

- Access is bound to **browser session**, not student identity cryptographically. Anyone with the victim's cookie can act as that exam session (standard web session threat model).
- Clearing cookies mid-exam requires resume with exam code again — acceptable UX tradeoff.

2. CSRF protection (A06-3) — detailed

Problem (technical)

HTML forms and `fetch()` POSTs did not include anti-CSRF tokens. Browsers send session cookies automatically on cross-origin requests subject to SameSite rules (Lax allows top-level POST navigation in some cases; embedded attacks and historical browser behavior still make tokens necessary).

Security concept

Synchronizer token pattern (OWASP recommended):

1. Server generates cryptographically random token per session.
2. Token embedded in every state-changing form (hidden field) and available to JavaScript via `<meta name="csrf-token">`.
3. Server validates token on POST before processing body.

Validation uses `secrets.compare_digest()` — **constant-time comparison** prevents attackers from guessing the token byte-by-byte via timing measurements.

Implementation

Component	File	Role
Token generation	<code>app/security.py</code> → <code>get_csrf_token()</code>	Creates/stores 32-byte url-safe token in session
Validation	<code>require_csrf(request, form_token)</code>	Called at start of every POST handler
Form field	<code>templates/includes/csrf_input.html</code>	<code><input type="hidden" name="csrf_token" ...></code>

Component	File	Role
JS access	templates/base.html	<meta name="csrf-token" content="...">
Hint AJAX	templates/question.html	Appends csrf_token to URLSearchParams
Client timing	templates/question.html	Same for /client-timing fetch
Config flag	RGEE_CSRF_ENABLED	Default true; false in test suite only

Forms updated (POST only):

- Student: /exam/start, /exam/start-nominated, /resume, answer, hint
- Instructor: login, logout, Together credentials save/clear, analysis feedback, nominated exam publish

GET forms (e.g. analysis filters) do not need CSRF — GET must be idempotent per HTTP semantics.

Why meta tag + hidden field?

- **Hidden field:** protects traditional form POSTs.
- **Meta tag:** allows JavaScript `fetch()` to read the token without parsing DOM forms — needed for hint carousel and performance timing beacons.

3. Security headers middleware (A02-1) — detailed

Security concept

Defense in depth at the browser layer. Even if an XSS flaw were introduced later, CSP restricts script execution sources. Headers apply uniformly via middleware so individual routes cannot forget them.

Implementation

Class: SecurityHeadersMiddleware in app/security.py

Registered: app.add_middleware(SecurityHeadersMiddleware) in app/main.py (runs on every response except static assets).

Header	Value (summary)	Attack mitigated
Content-Security-Policy	default-src 'self'; scripts from self + cdnjs; styles from self + Google Fonts + jsDelivr	XSS, unauthorized resource load
X-Frame-Options	DENY	Clickjacking
X-Content-Type-Options	nosniff	MIME confusion
Referrer-Policy	strict-origin-when-cross-origin	URL path leakage
Permissions-Policy	camera/mic/geo/payment disabled	Unneeded feature abuse
Strict-Transport-Security	max-age=31536000; includeSubDomains	SSL stripping (when HTTPS enabled)

CSP tradeoff: 'unsafe-inline' is required for existing inline scripts (theme bootstrap, exam JS). Tightening further would require refactoring to external script files with nonces — a future hardening step.

4. Production security gate (A02-3, A07-1) — detailed

Security concept

Fail closed / secure defaults: Production deployments should refuse to start with known-weak configuration rather than silently inheriting development values.

Implementation

Function: `validate_production_security(settings)` — called in application lifespan startup.

When `RGEE_PRODUCTION=1`, startup raises **RuntimeError** if:

Condition	Reason
<code>INSTRUCTOR_SESSION_SECRET</code> equals dev default	Cookie forgery risk
<code>RGEE_CSRF_ENABLED</code> is false	CSRF protection mandatory in production
Analysis agent URL set but secret empty	Unauthenticated internal API

Instructor credentials (`app/instructor_auth.py`):

- `ensure_instructor_credentials_file()` raises in production if file missing — operator must provision hashes deliberately.
- In development, creates file with known test hashes and logs **WARNING**.

Environment variables (new)

Variable	Default	Purpose
<code>RGEE_PRODUCTION</code>	0	Enables strict startup checks
<code>RGEE_CSRF_ENABLED</code>	1	Toggle CSRF (tests use 0)
<code>INSTRUCTOR_SESSION_HTTPS_ONLY</code>	0	Cookie Secure flag

Documented in `.env.example`.

5. Input length limits (A05-4) — detailed

Security concept

Input validation as abuse prevention — limits damage from oversized payloads (DoS, storage exhaustion, LLM token burning). Client-side `maxLength` attributes exist in some templates but are **not authoritative**; server-side enforcement is required (OWASP Input Validation).

Implementation

Constants in `app/security.py`:

Constant	Value	Applied to
<code>MAX_STUDENT_ID_LEN</code>	256	Start, resume, nominated start
<code>MAX_PROFESSOR_DOMAIN_LEN</code>	8,000	Exam start domain field
<code>MAX_ANSWER_LEN</code>	50,000	Answer submission

Helper: `truncate_field(value, max_len)` strips whitespace and truncates — rejects unbounded storage while preserving student content up to a generous essay limit.

6. Session fixation fix (A07-2) — detailed

Before

```
request.session[SESSION_KEY] = True # Reused existing session ID
```

After

```
request.session.clear()  
request.session[SESSION_KEY] = True # New session ID after authentication
```

Concept

After login, the session identifier should rotate so a pre-login session ID planted by an attacker cannot be elevated to an authenticated session.

7. Broken HTML hint endpoint (A10-1) — detailed

Problem

The no-JavaScript fallback form posted to `/exam/{id}/hint`, but the handler referenced undefined Python variables copied from the JSON hint path — causing unhandled `NameError` → 500 page (safely generic, but broken functionality).

Fix

Unified fallback to `generate_safe_hint()` — same safe code path conceptually aligned with hint mode in `/hint-json`.

Concept

Consistent error handling and code path parity — exceptional conditions (no JS) should not hit undeveloped code branches (A10 Mishandling of Exceptional Conditions).

8. Analysis agent hardening (A01-3, A02-4) — detailed

Changes

`RGEE_Analysis_Agent/app/main.py`:

```
if not secret and get_settings().require_api_secret:  
    raise HTTPException(503, "Analysis agent is not configured with RGEE_ANALYSIS_AGENT_SECRET.")
```

`docker-compose.yml`:

- Port mapping: 127.0.0.1:8010:8010 (not 0.0.0.0)
- Both services receive matching `RGEE_ANALYSIS_AGENT_SECRET`
- Agent sets `RGEE_ANALYSIS_AGENT_REQUIRE_SECRET=1`

Concept

Network segmentation + shared secret authentication for internal microservices. The agent should not be reachable from the public internet; the main app proves identity via `X-RGEE-Analysis-Secret` header on each request.

9. HTTPS-only session cookies (A02-2) — detailed

Starlette `SessionMiddleware` accepts `https_only=settings.session_https_only`.

When `True`, the session cookie includes the **Secure attribute** — browsers will not send it over HTTP. Use in production behind TLS termination.

10. Automated security tests — detailed

Test file	Test	What it proves
tests/security/test_controls.py	test_exam_routes_require_session_match_token	103 IDOR session match token
tests/security/test_controls.py	test_security_headers_are_present	attaches headers
tests/security/test_controls.py	test_csrf_blocks_instructor_login_without_token	CSRF for login without token
tests/security/test_http.py	Traceback tests	No stack traces in HTML errors
tests/conftest.py	RGEE_CSRF_ENABLED=0	Existing integration tests unchanged

Test credentials: `conftest.py` pre-writes `instructor_credentials.json` so tests do not depend on auto-created production-blocked files.

File change index

File	Change summary
app/security.py	New — CSRF, exam access, headers, limits, production validation
app/main.py	Middleware, guards on routes, session clear on login, hint fix
app/config.py	RGEE_PRODUCTION, RGEE_CSRF_ENABLED, INSTRUCTOR_SESSION_HTTPS_ONLY
app/instructor_auth.py	Production credential file policy
templates/includes/csrf_input.html	New — reusable CSRF hidden input
templates/base.html	CSRF meta tag
templates/*.html	CSRF includes on POST forms; JS token in question page
RGEE_Analysis_Agent/app/main.py	Require secret mode
RGEE_Analysis_Agent/app/conftest.py	RGEE_ANALYSIS_AGENT_REQUIRE_SECRET
docker-compose.yml	Localhost bind + shared secrets
.env.example	Documented new variables
tests/conftest.py	Test creds file, CSRF disabled for suite
tests/security/test_controls.py	New security control tests

Deployment checklist (production)

```
# 1. Environment (minimum)
RGEE_PRODUCTION=1
INSTRUCTOR_SESSION_SECRET=<use: openssl rand -base64 48>
INSTRUCTOR_SESSION_HTTPS_ONLY=1
RGEE_CSRF_ENABLED=1
MOCK_LLM=0    # if using real Together API

# 2. Instructor credentials - create BEFORE first boot:
#   instructor_credentials.json with username_sha256 + password_pbkdf2_hex
#   (never use default elliot/12345 on a public server)
```

```
# 3. Analysis agent (if used)
RGEE_ANALYSIS_AGENT_URL=http://127.0.0.1:8010
RGEE_ANALYSIS_AGENT_SECRET=<shared random string>
RGEE_ANALYSIS_AGENT_REQUIRE_SECRET=1

# 4. Reverse proxy
#   TLS termination, rate limiting on /professor/login and /resume

# 5. Verify
python -m pytest tests/ -q
```

Verification and test evidence

```
$ python -m pytest tests/ -q
66 passed in ~31s
```

Security-relevant tests span:

- tests/security/ — traceback leakage, IDOR, headers, CSRF
 - tests/general/test_api.py — end-to-end exam flows with access tokens (via shared session in TestClient)
 - tests/general/test_instructor_auth.py — login verification edge cases
-

References

Primary standards

- OWASP Top 10:2025 — Broken Access Control, Security Misconfiguration, Injection, Authentication Failures, etc.
- OWASP ASVS 4.0 — Verification requirements for web applications
- OWASP Cheat Sheet Series — CSRF, Session Management, Input Validation, HTTP Headers

Platform and framework

- Mozilla Web Security Guidelines — CSP, HSTS, cookie guidance
- FastAPI Security documentation
- Starlette SessionMiddleware

Weakness classification

- CWE Top 25 — CWE-639 (IDOR), CWE-352 (CSRF), CWE-798 (hard-coded credentials)
- CWE-384 Session Fixation

LLM-specific

- OWASP Top 10 for LLM Applications — prompt injection, excessive agency

Authentication

- NIST SP 800-63B — password hashing and verifier requirements
-

Report prepared for team review. Part 1 documents findings; Part 2 documents remediation with concepts and implementation detail for maintainers.